
Advanced Data Mining

Data Mining Techniques

Content - Common techniques in data mining

- ☐ Association rules
 - ☐ Classification
 - ☐ Clustering
 - ☐ Sequence Patterns
-

Association rules

Introduction

- ❑ Many retail stores collect data about customers.
- ❑ e.g. customer transactions
- ❑ Need to analyze this data to understand customer behavior
- ❑ Why?
 - for marketing purposes,
 - inventory management,
 - customer relationship management

Introduction

Discovering patterns and associations

- ❑ Discovering interesting relationships hidden or frequent pattern (a pattern is a set of items, subsequences, substructures, etc.) that occurs frequently in large databases.
 - beer and diapers are often sold together
 - ❑ **pattern mining** is a fundamental data mining problem with many applications in various fields.
 - ❑ First introduced by Agrawal (1993) in the context of **frequent itemsets** and **association rule mining**
 - ❑ It is an important data mining model studied extensively by the database and data mining community.
 - ❑ Many extensions of this problem to discover patterns in graphs, sequences, and other kinds of data.
-

Frequent itemset mining

Definitions

□ Let $I = \{I_1, I_2, I_3, \dots, I_n\}$ be the set of items (products) that appear (sold) in the database (in a retail store).

e.g., $I = \{\text{pasta, lemon, bread, orange, cake}\}$

□ A **transaction database** D is a set of transactions.

$D = \{T_1, T_2, \dots, T_r\}$ such that $T_a \subseteq I$ ($1 \leq a \leq r$).

Number of transaction in D denoted $|D|$

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

□ Each transaction has a unique identifier called its **Transaction ID** (TID).

e.g. the transaction ID of T4 is 4.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

A transaction is a set of items (an *itemset*).

e.g. T2= {pasta, lemon}

An **item** (a symbol) may not appear or appear once in each transaction.
Each transaction is unordered.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

A transaction database can be viewed as a binary matrix:

Transaction	pasta	lemon	bread	orange	cake
T1	1	1	1	1	0
T2	1	1	0	0	0
T3	1	0	0	1	1
T4	1	1	0	1	1

- Asymetrical binary attributes (because 1 is more important than 0)
 - There is no information about purchase quantities and prices.
-

Definitions

Let **I** be the set of all items:

$I = \{\text{pasta, lemon, bread, orange, cake}\}$

There are $2^{|I|} - 1 = 2^5 - 1 = 31$ subsets :

$\{\text{pasta}\}, \{\text{lemon}\}, \{\text{bread}\}, \{\text{orange}\}, \{\text{cake}\}$

$\{\text{pasta, lemon}\}, \{\text{pasta, bread}\}, \{\text{pasta, orange}\}, \{\text{pasta, cake}\}, \{\text{lemon, bread}\}, \{\text{lemon orange}\},$

$\{\text{lemon, cake}\}, \{\text{bread, orange}\}, \{\text{bread cake}\}$

...

$\{\text{pasta, lemon, bread, orange, cake}\}$

Definitions

□ **Itemset** is a set of one or more items

- E.g.: {pasta, lemon, bread}

□ An **itemset** is said to be **of size k** , if it contains k items.

- Itemsets of size 1:

{pasta}, {lemon}, {bread}, {orange}, {cake}

- Itemsets of size 2:

{pasta, lemon}, {pasta, bread}, {pasta, orange}, {pasta, cake}, {lemon, bread}, {lemon orange}, ...

Definitions

The **support (frequency)** of an itemset **X** is the number of transactions that contains **X** (Frequency of occurrence of an itemset).

$$\text{sup}(X) = |\{T \mid X \subseteq T \wedge T \in D\}|$$

For example: The support of {pasta, orange} is 3.

which is written as: $\text{sup}(\{\text{pasta, orange}\}) = 3$

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

The **support of an itemset X** can also be written as a ratio (*absolute support*). Fraction of the transactions that contain an itemset.

Example: The support of {pasta, orange} is 75% because it appears in 3 out of 4 transactions.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

- ❑ **Frequent Itemset:** An itemset whose support is greater than or equal to a **minsup** (minimum support) threshold, $\text{sup}(X) \geq \text{minsup}$
- ❑ Let there be a numerical value **minsup**, set by the user.
- ❑ For example:
Given **minsup** = 2 and $\text{sup}(\{\text{pasta}, \text{orange}\}) = 3$
{pasta, orange} is a Frequent Itemset

The problem of frequent itemset mining

- Frequent itemset mining (FIM) consists of enumerating all **frequent itemsets**, that is itemsets having a support greater or equal to **minsup**.
- **Input**: A set of transactions **T**, over a set of items **I**, **minsup** value
- **Output**: All itemsets with items in **I** having $\text{sup}(I) \geq \text{minsup}$

Example

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange cake}

For **minsup = 2**, the frequent itemsets are:

{lemon}, {pasta}, {orange}, {cake}, {lemon, pasta}, {lemon, orange}, {pasta, orange},
{pasta, cake}, {orange, cake}, {lemon, pasta, orange}, {pasta, orange, lemon}

The problem of frequent itemset mining

□ Example: Let $I = \{A, B, C, D, E, F\}$ and a transaction database as:

T_1	$\{A, B, C, D\}$
T_2	$\{A, C, E\}$
T_3	$\{A, E\}$
T_4	$\{A, E, F\}$
T_5	$\{A, B, C, E, F\}$

With $X = \{A, E\}$, $minsup=60\%$

$$\rightarrow sup(X) = 4$$

$$or\ sup(X) = 4/5 = 80\%$$

$\rightarrow X$ is frequent itemset

The problem of frequent itemset mining

Given $I = \{ \text{Beer, Bread, Jelly, Milk, PeanutButter} \}$ and a transaction database as:

TID	Items
10	Bread, Jelly, PeanutButter
20	Bread, PeanutButter
30	Bread, Milk, PeanutButter
40	Beer, Bread
50	Beer, Milk

minsup = 60%

Find the support for each below itemset and which itemset is frequent itemset

$X_1 = \{ \text{Bread, PeanutButter} \}$

$X_2 = \{ \text{Bread} \}$

$X_3 = \{ \text{PeanutButter} \}$

$X_4 = \{ \text{Milk} \}$

$X_5 = \{ \text{Milk, Bread} \}$

Association rule mining

- ❑ A **data analysis** task proposed by Agrawal & Srikant (1993)
 - ❑ Finding frequent patterns in a database allows to find useful information.
 - E.g. discover the products that people like to purchase together frequently
 - ❑ But it has some limitations
 - ❑ **Goal:** find interesting associations between values in a dataset
-

Introduction

A **transactional database** D

Transaction	Items in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

If $\text{minsup} = 2$, then {pasta, cake} is frequent.

Can we conclude that people who buy pasta will also buy cakes?

Association rule mining

Applications:

- ❑ Market Basket Analysis

Marketing & Retail. E.g., frequent itemsets give information about "other customer who bought this item also bought X"

- ❑ Exploratory Data Analysis

Find correlation in very large (= many transactions), high-dimensional (= many items) data

- ❑ Intrusion Detection

Rules with low support but very high lift

- ❑ Build Rule-based Classifiers

Class association rules (CARs)

Association rule mining

- ❑ Foundation for many essential data mining tasks
 - Association, correlation, and causality analysis
 - Sequential, structural (e.g., sub-graph) patterns
 - Pattern analysis in spatiotemporal, multimedia, time-series, and stream data
 - Classification: discriminative, frequent pattern analysis
 - Cluster analysis: frequent pattern-based clustering
 - Data warehousing: iceberg cube and cube-gradient
 - Semantic data compression: fascicles
 - Broad applications
-

Basic Concepts: Association Rules

Let **D** be database of transactions

– e.g.:

Transaction ID	Items
1000	A, B, C
2000	A, B
3000	A, D
4000	B, E, F

- Let **I** be the set of items that appear in the database, e.g., **I**={A,B,C,D,E,F}
 - Each transaction **t** is a subset of **I**
- A **rule** is an implication among **itemsets** **X** and **Y**, of the form by **X** $\rightarrow$ **Y**, where **X**⊂**I**, **Y**⊂**I**, and **X**∩**Y**=∅
 - e.g.: **{B,C}** $\rightarrow$ **{A}** is a rule

Basic Concepts: Association Rules

□ Association Rule

- An implication expression of the form $X \rightarrow Y$, where X and Y are non-overlapping itemsets
- $\{\text{Milk, Diaper}\} \rightarrow \{\text{Beer}\}$

□ Rule Evaluation Metrics

■ Support (s)

- Fraction of transactions that contain both X and Y
- i.e., support of the itemset $X \cup Y$

■ Confidence (c)

- Measures how often items in Y appear in transactions that contain X

$$c(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)} = \frac{s(X \cup Y)}{s(X)}$$

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Example:

$\{\text{Milk, Diaper}\} \rightarrow \text{Beer}$

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{|D|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

Basic Concepts: Association Rules

- Another interpretation of support and confidence for $X \rightarrow Y$
 - **Support** is the probability that a transaction contains $\{X \cup Y\}$ or $\Pr(X \wedge Y)$

$$\text{support}(X \rightarrow Y) = \text{support}(X \cup Y) = \sigma(X \cup Y) / |T|$$

- **Confidence** is the *conditional probability* that a transaction will contains Y given that it contains X or $\Pr(Y | X)$

$$\begin{aligned} \text{confidence}(X \rightarrow Y) &= \sigma(X \cup Y) / \sigma(X) \\ &= \text{support}(X \cup Y) / \text{support}(X) \end{aligned}$$

Support and Confidence - Example

$$\text{support}(X \rightarrow Y) = \text{support}(X \cup Y) = \sigma(X \cup Y) / |D|$$

$$\begin{aligned}\text{confidence}(X \rightarrow Y) &= \sigma(X \cup Y) / \sigma(X) \\ &= \text{support}(X \cup Y) / \text{support}(X)\end{aligned}$$

Transaction ID	Items Bought
1001	A, B, C
1002	A, C
1003	A, D
1004	B, E, F
1005	A, D, F



Itemset {A, C} has a support of $2/5 = 40\%$

Rule {A} $\Rightarrow$ {C} has confidence of 50%

Rule {C} $\Rightarrow$ {A} has confidence of 100%

Support for {A, C, E} ?

Support for {A, D, F} ?

Confidence for {A, D} $\Rightarrow$ {F} ?

Confidence for {A} $\Rightarrow$ {D, F} ?

Association Rule Mining Task

- ❑ Given a set of transactions, the goal of association rule mining is to find all rules having that satisfy the user-specified *minimum support* (minsup) and *minimum confidence* (minconf):
 - $\text{support} \geq \text{minsup threshold}$
 - $\text{confidence} \geq \text{minconf threshold}$
- ❑ **Brute Force Algorithm:**
 - List all possible itemsets and compute their support
 - Generate all rules from frequent itemset
 - Prune rules that fail the minconf threshold

Would this work?!

→ Computationally prohibitive!

Mining Association Rules

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Example of Rules:

$\{\text{Milk, Diaper}\} \rightarrow \{\text{Beer}\}$ ($s=0.4$, $c=0.67$)

$\{\text{Milk, Beer}\} \rightarrow \{\text{Diaper}\}$ ($s=0.4$, $c=1.0$)

$\{\text{Diaper, Beer}\} \rightarrow \{\text{Milk}\}$ ($s=0.4$, $c=0.67$)

$\{\text{Beer}\} \rightarrow \{\text{Milk, Diaper}\}$ ($s=0.4$, $c=0.67$)

$\{\text{Diaper}\} \rightarrow \{\text{Milk, Beer}\}$ ($s=0.4$, $c=0.5$)

$\{\text{Milk}\} \rightarrow \{\text{Diaper, Beer}\}$ ($s=0.4$, $c=0.5$)

Observations:

- All the above rules are binary partitions of the same itemset: {Milk, Diaper, Beer}
- Rules originating from the same itemset have identical support but can have different confidence
- Thus, we may decouple the support and confidence requirements

Association Rules Mining

□ Two-step approach:

1. Frequent Itemset Generation

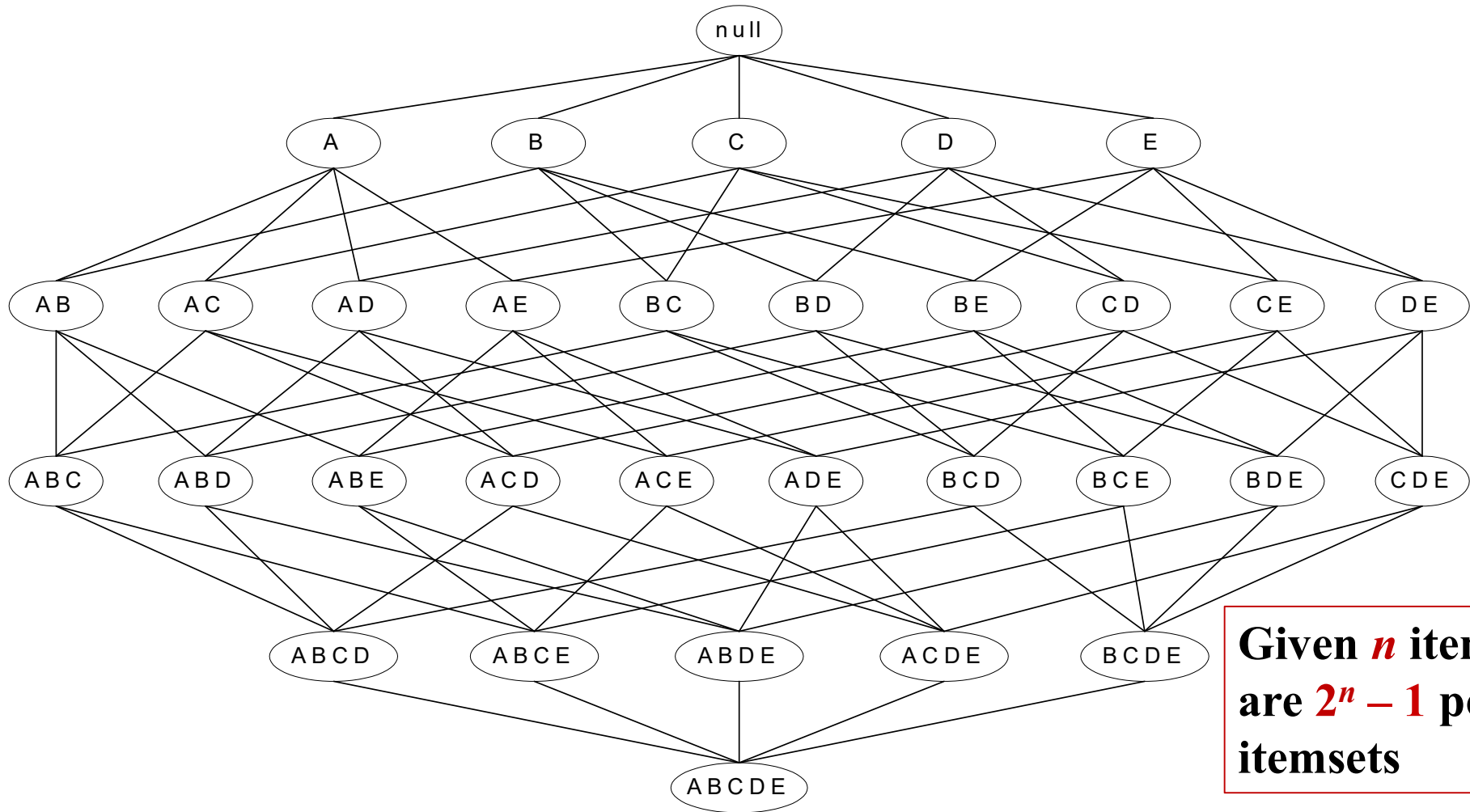
- Generate all itemsets whose support $\geq$ minsup

2. Rule Generation

- Generate high confidence rules from each frequent itemset, where each rule is a binary partitioning of a frequent itemset

□ Frequent itemset generation is still computationally expensive

How many itemsets are there?



Given n items, there are $2^n - 1$ possible itemsets

Generating Association Rules from Frequent Itemsets

Given a frequent itemset F

- Find all non-empty subsets $f \subset F$
- For each non-empty subsets $f \subset F$, generate a candidate association rule such that $f \rightarrow F - f$
- A candidate association rule $A \rightarrow B$ is an association rule (strong rule) if its confidence satisfies the minimum confidence requirement

$$\text{confidence}(A \rightarrow B) = \Pr(B \mid A) = \frac{\text{support}(A \cup B)}{\text{support}(A)}$$

- If $|F| = k$, then there are $2^k - 2$ candidate association rules (ignoring $L \rightarrow \emptyset$ and $\emptyset \rightarrow L$)

Generating Association Rules from Frequent Itemsets

Example: If $X=\{A,B,C,D\}$ is a frequent itemset

- Non-empty subsets $f \subset X$: $ABC, ABD, ACD, BCD, A, B, C, D, AB, AC, AD, BC, BD, CD$
- Candidate rules generated from X :
 - $ABC \rightarrow D, ABD \rightarrow C, ACD \rightarrow B, BCD \rightarrow A,$
 - $A \rightarrow BCD, B \rightarrow ACD, C \rightarrow ABD, D \rightarrow ABC$
 - $AB \rightarrow CD, AC \rightarrow BD, AD \rightarrow BC, BC \rightarrow AD,$
 - $BD \rightarrow AC, CD \rightarrow AB$

Frequent itemset mining Techniques

Introduction

Discovering patterns and associations

- ❑ Discovering interesting relationships hidden in large databases.
 - ❑ e.g. **beer and diapers are often sold together**
 - ❑ **pattern mining** is a fundamental data mining problem with many applications in various fields.
 - ❑ Introduced by Agrawal (1993).
 - ❑ Many extensions of this problem to discover patterns in graphs, sequences, and other kinds of data.
-

Numerous applications

Frequent itemset mining has numerous applications.

- medical applications,
- chemistry,
- biology,
- e-learning,
- etc.

Definitions

Let $I = \{I_1, I_2, \dots, I_m\}$ be the set of **items** (products) sold in a retail store.

For example:

$I = \{\text{pasta, lemon, bread, orange, cake}\}$

Definitions

❑ Let $I = \{I_1, I_2, \dots, I_m\}$ be the set of **items** (products) sold in a retail store.

For example: $I = \{\text{pasta, lemon, bread, orange, cake}\}$

❑ A **transaction database** D is a set of **transactions**.

$D = \{T_1, T_2, \dots, T_r\}$ such that $T_a \subseteq I$ ($1 \leq a \leq r$).

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

Each transaction has a unique identifier called its *Transaction ID (TID)*.

e.g. the transaction ID of T_4 is 4.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

A transaction is a set of items (an *itemset*).

e.g. $T2 = \{\text{pasta, lemon}\}$

An **item** (a symbol) may not appear or appear once in each transaction. Each transaction is unordered.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

A transaction database can be viewed as a binary matrix:

Transaction	pasta	lemon	bread	orange	cake
T1	1	1	1	1	0
T2	1	1	0	0	0
T3	1	0	0	1	1
T4	1	1	0	1	1

- Asymmetrical binary attributes (because 1 is more important than 0)
 - There is no information about purchase quantities and prices.
-

Definitions

Let **I** be the set of all items:

$I = \{\text{pasta, lemon, bread, orange, cake}\}$

There are $2^{|I|} - 1 = 2^5 - 1 = 31$ subsets :

$\{\text{pasta}\}, \{\text{lemon}\}, \{\text{bread}\}, \{\text{orange}\}, \{\text{cake}\}$

$\{\text{pasta, lemon}\}, \{\text{pasta, bread}\}, \{\text{pasta, orange}\}, \{\text{pasta, cake}\}, \{\text{lemon, bread}\},$
 $\{\text{lemon, orange}\},$

$\{\text{lemon, cake}\}, \{\text{bread, orange}\}, \{\text{bread, cake}\}$

...

$\{\text{pasta, lemon, bread, orange, cake}\}$

Definitions

An **itemset** is said to be **of size k** , if it contains k items.

Itemsets of size 1:

{pasta}, {lemon}, {bread}, {orange}, {cake}

Itemsets of size 2:

{pasta, lemon}, {pasta, bread}, {pasta, orange}, {pasta, cake}, {lemon, bread},
{lemon orange}, ...

Definitions

The **support (frequency)** of an itemset **X** is the number of transactions that contains **X**.

$$\text{sup}(X) = |\{T \mid X \subseteq T \wedge T \in D\}|$$

For example: The support of {pasta, orange} is 3.

which is written as: $\text{sup}(\{\text{pasta}, \text{orange}\}) = 3$

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Definitions

The **support of an itemset X** can also be written as a ratio (*absolute support*).

Example: The support of {pasta, orange} is 75% because it appears in 3 out of 4 transactions.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

The problem of frequent itemset mining

- Let there be a numerical value *minsup*, set by the user.
- Frequent itemset mining (FIM) consists of enumerating all **frequent itemsets**, that is itemsets having a support greater or equal to *minsup*.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

Example

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange cake}

For *minsup* = 2, the **frequent itemsets** are:

{lemon}, {pasta}, {orange}, {cake}, {lemon, pasta}, {lemon, orange}, {pasta, orange}, {pasta, cake}, {orange, cake}, {lemon, pasta, orange}

For the user, choosing a high *minsup* value,

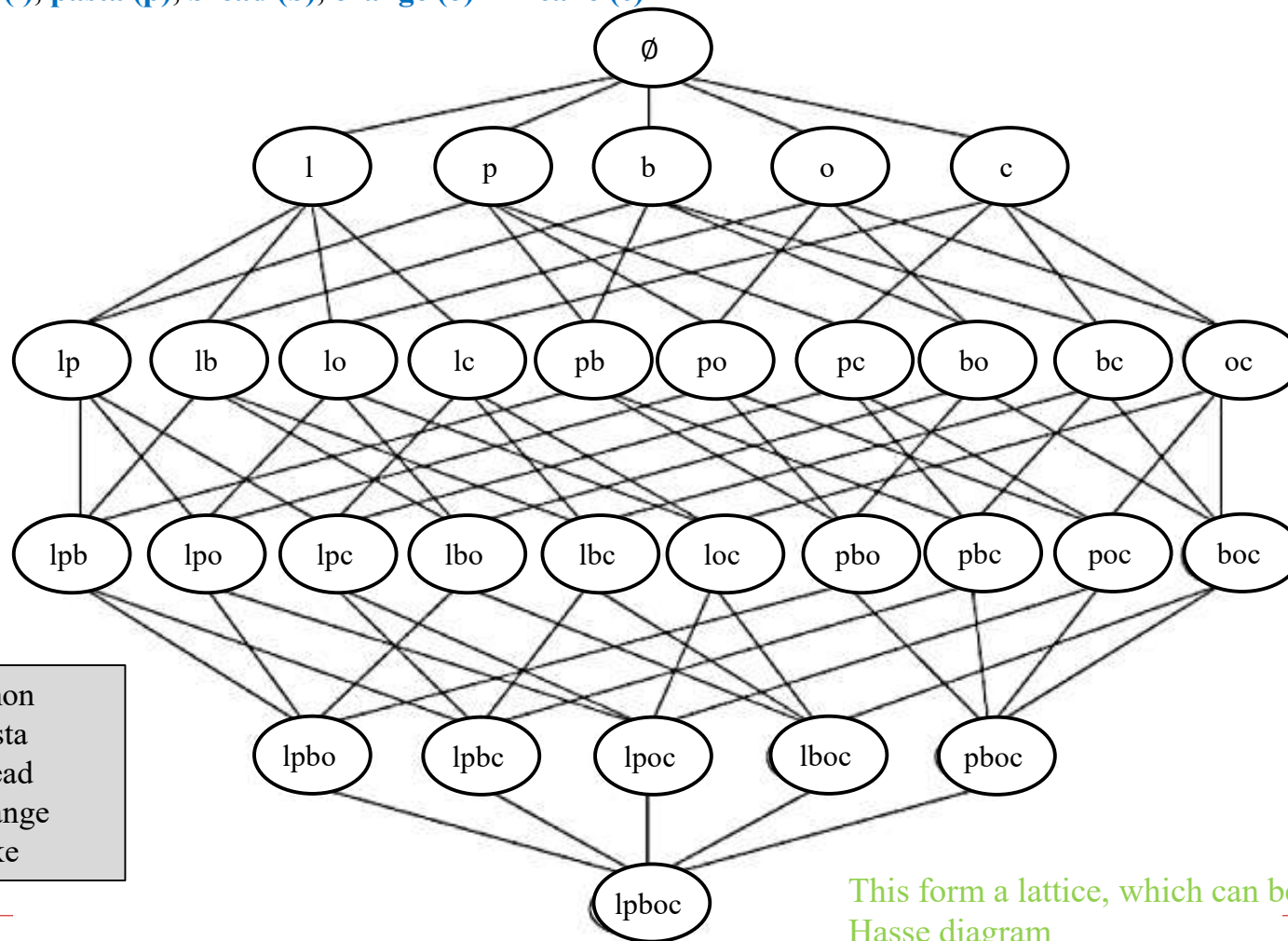
- ☐ will reduce the number of frequent itemsets,
- ☐ will increase the speed and decrease the memory required for finding the frequent itemsets

Naïve approach

- ❑ If there are n items in a database, there are $2^n - 1$ itemsets may be frequent.
- ❑ **Naïve approach:** count the support of all these itemsets.
- ❑ To do that, we would need to read each transaction in the database to count the support of each itemset.
- ❑ This would be inefficient:
 - need to perform too many comparisons
 - requires too much memory

Search space

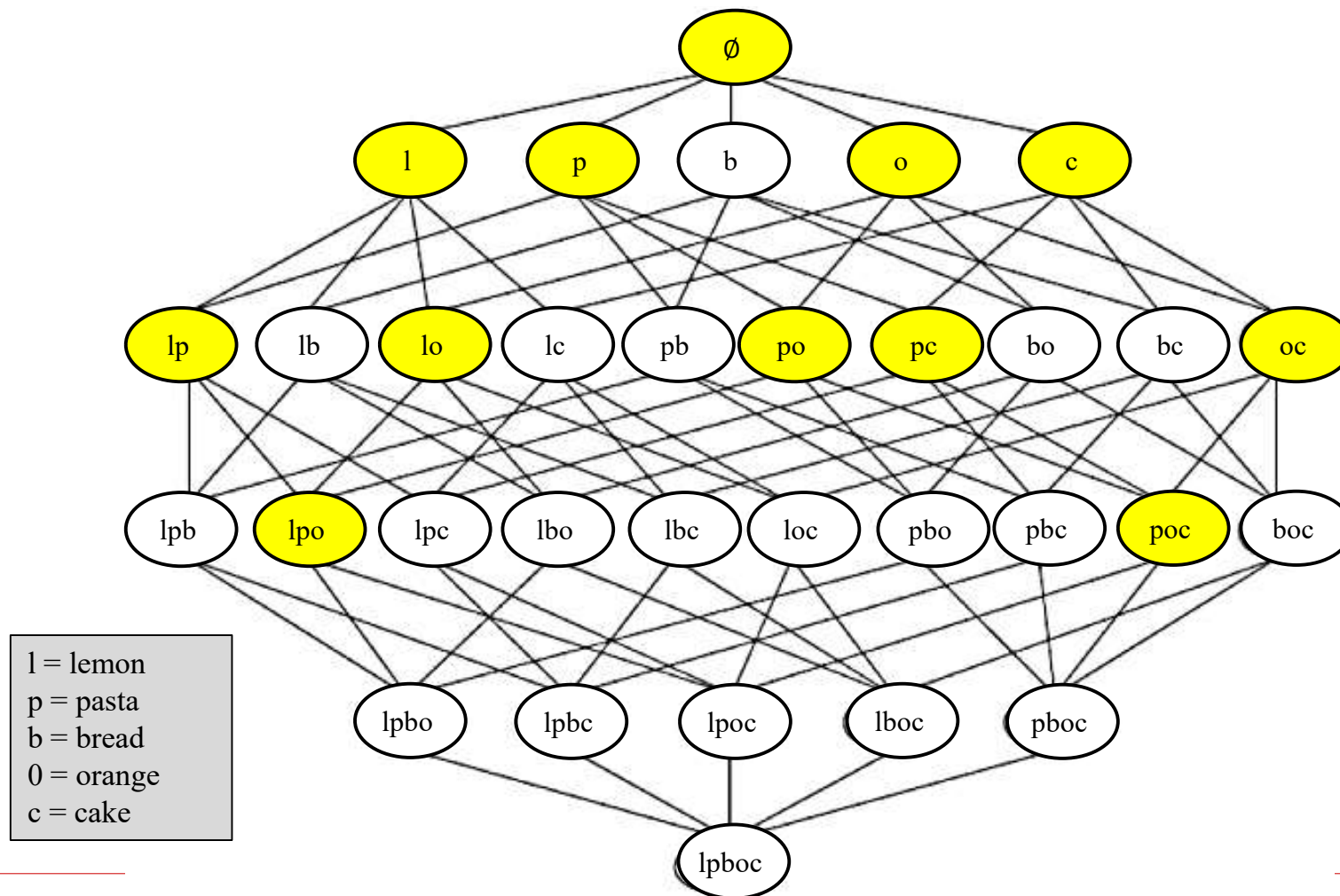
This is all the itemsets that can be formed with the items
lemon (l), pasta (p), bread (b), orange (o) and cake (c)



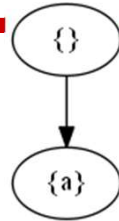
This form a lattice, which can be viewed as a
Hasse diagram

Search space

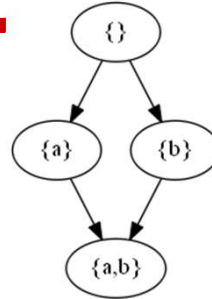
If **minsup** = 2, the frequent itemsets are (in yellow):



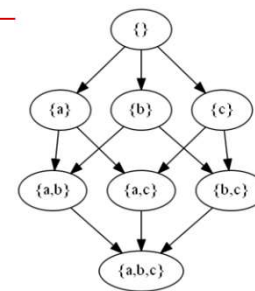
$I=\{A\}$



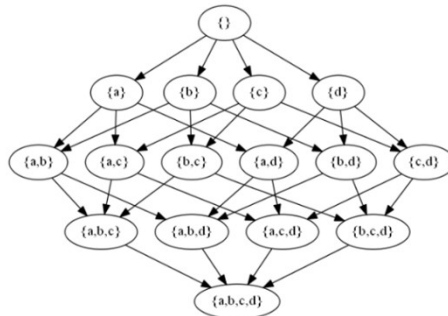
$I=\{A, B\}$



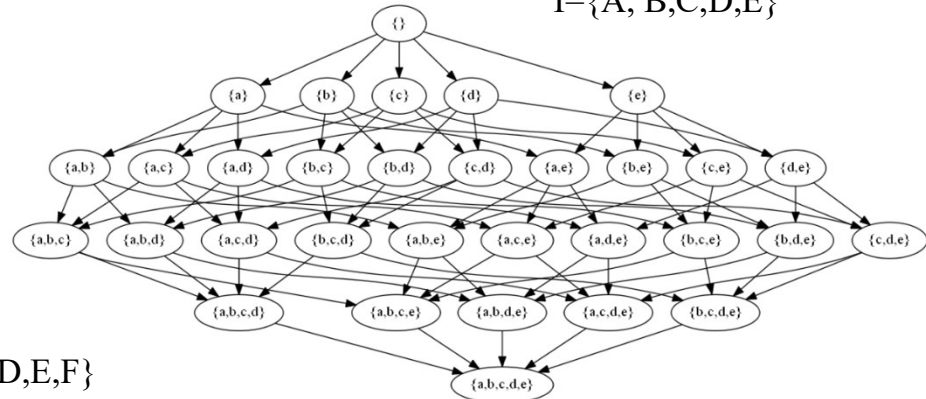
$I=\{A, B, C\}$



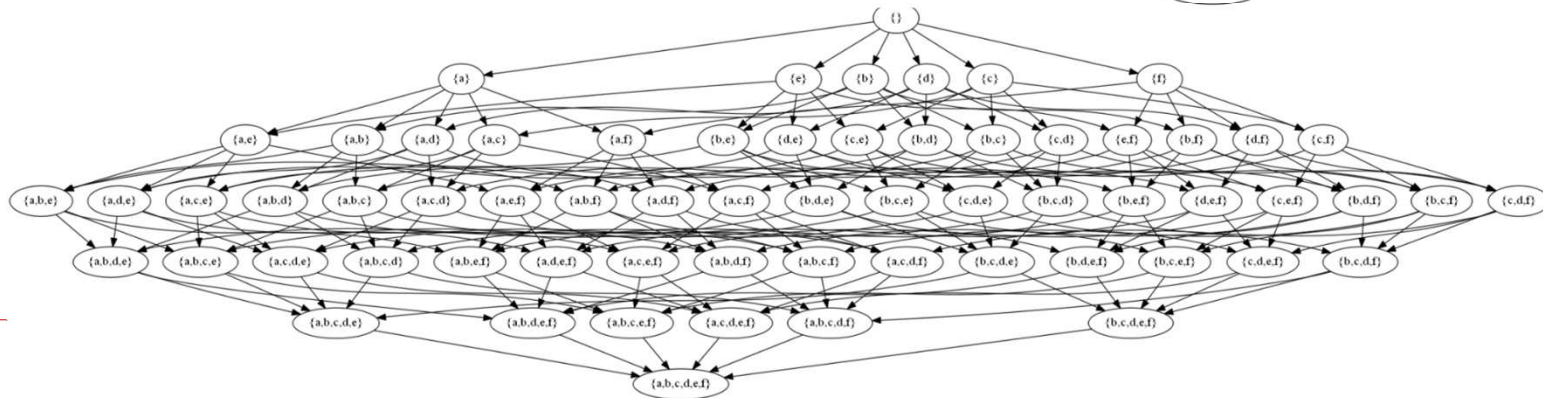
$I=\{A, B, C, D\}$



$I=\{A, B, C, D, E\}$



$I=\{A, B, C, D, E, F\}$



How to find the frequent itemsets?

Two challenges:

- ☐ How to count the support of itemsets in an efficient way (not spend too much time or memory)?
- ☐ How to reduce the search space (we do not want to consider all the possibilities)?

Several algorithms

- Algorithms:
 - **Apriori**, AprioriTID
 - Eclat
 - **FPGrowth**
 - **Hmine**
 - **LCM**, ...
 - ...
- Moreover, numerous extensions of the FIM problem: uncertain data, fuzzy data, purchase quantities, profit, weight, time, rare itemsets, closed itemsets, etc.

The APRIORI algorithm (Agrawal & Srikant, 1993/1994)

*R. Agrawal and R. Srikant. Fast algorithms for mining
association rules in large databases. Research Report
RJ 9839, IBM Almaden Research Center, San Jose,
California, June 1994.*

Introduction

Apriori is a famous algorithm

- ❑ which is not the most efficient algorithm,
- ❑ but has inspired many other algorithms!
- ❑ has been applied in many fields,
- ❑ has been adapted for many other similar problems.

Apriori is based on two important properties

Apriori property: Let there be two itemsets X and Y .
If $X \subset Y$, the support of Y is less than or equal to the support of X .

Example:

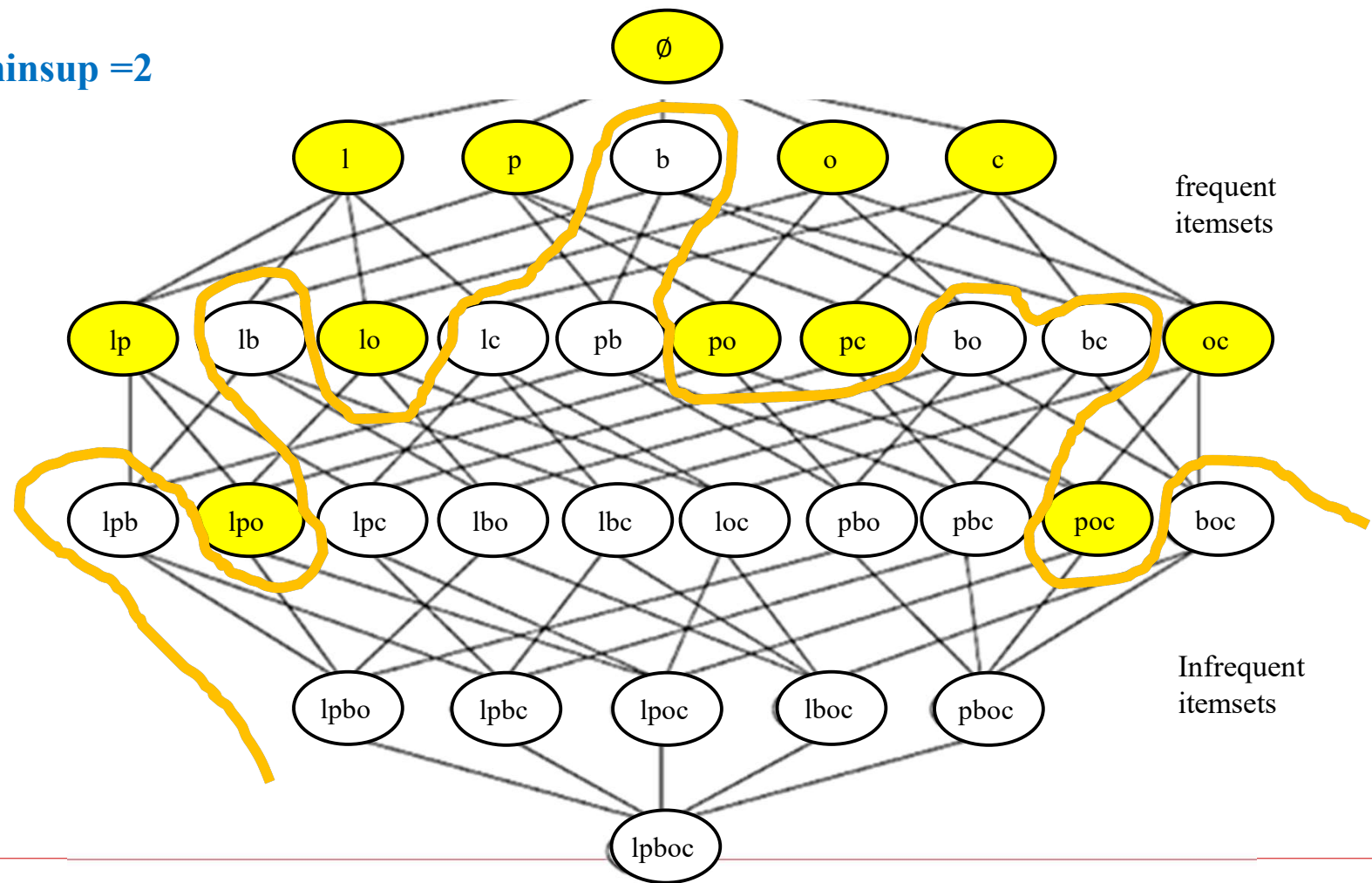
- The support of {pasta} is 4
- The support of {pasta, lemon} is 3
- The support of {pasta, lemon, orange} is 2

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

(support is anti-monotonic)

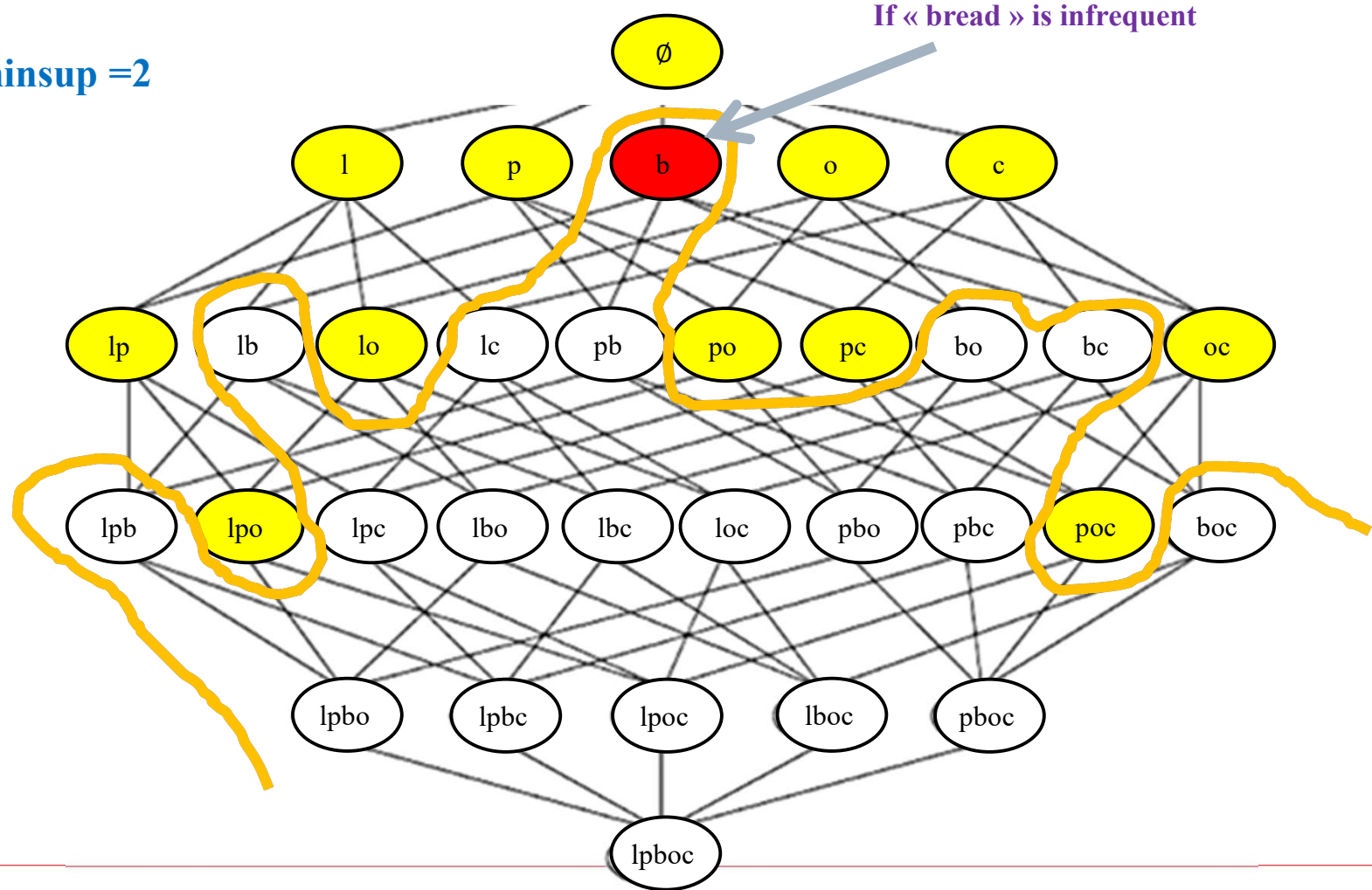
Illustration

minsup = 2



This property is useful to reduce the search space. **Example:**

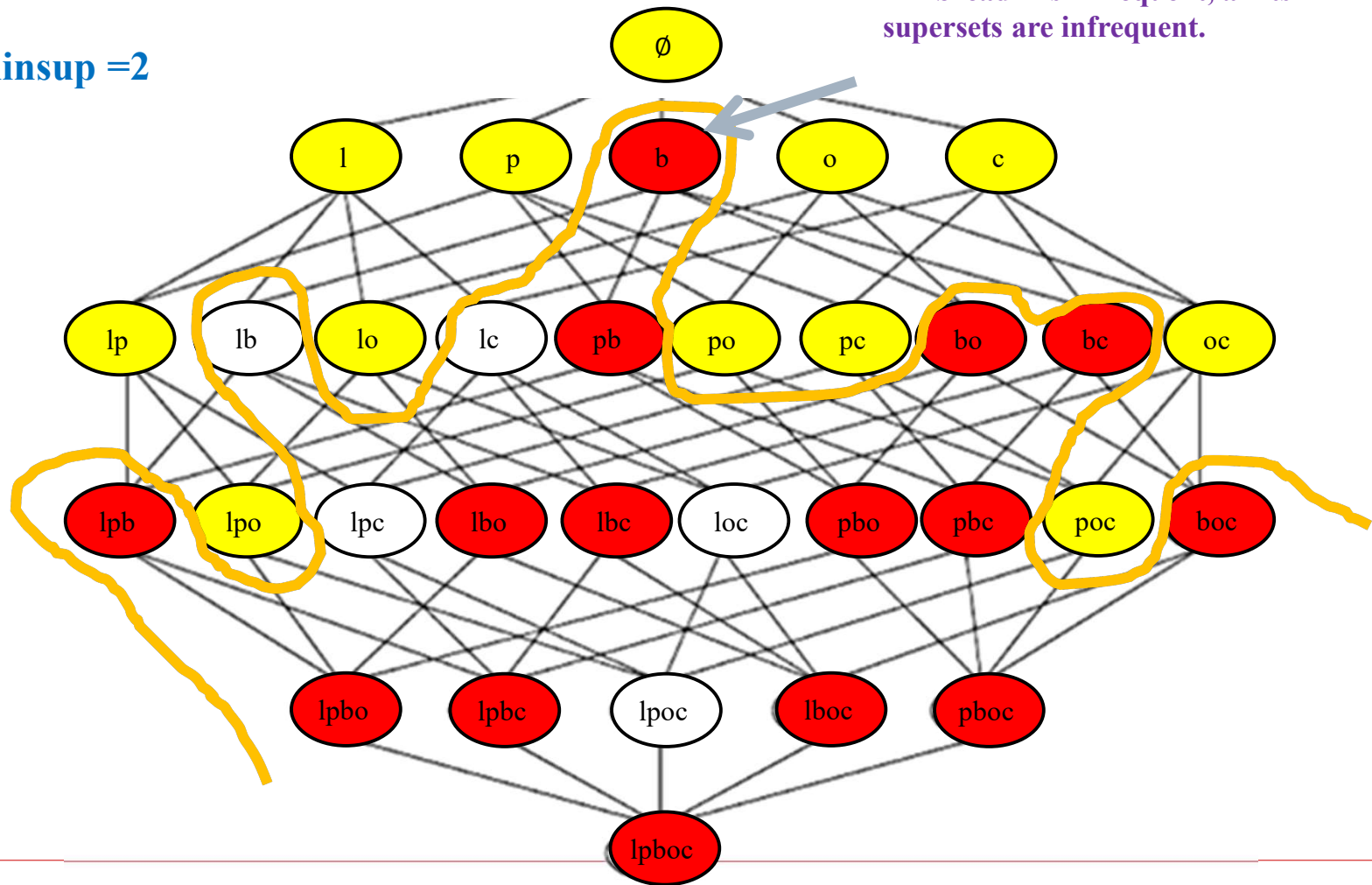
minsup = 2



This property is useful to reduce the search space. **Example:**

minsup = 2

If « bread » is infrequent, all its supersets are infrequent.



Property 2: Let there be an itemset Y .

If there exists an itemset $X \subset Y$ such that X is infrequent, then Y is infrequent.

Example:

- Consider $\{\text{bread, lemon}\}$.
- If we know that $\{\text{bread}\}$ is infrequent, then we can infer that $\{\text{bread, lemon}\}$ is also infrequent.

Transaction	Items appearing in the transaction
T1	{pasta, lemon, bread, orange}
T2	{pasta, lemon}
T3	{pasta, orange, cake}
T4	{pasta, lemon, orange, cake}

The Apriori algorithm

- I will now explain how the Apriori algorithm works
- **Input:**
 - *minsup*
 - a transactional database
- **Output:**
 - all the frequent itemsets

Consider *minsup* = 2.

The Apriori algorithm

Step 1: scan the database to calculate the support of all itemsets of size 1.

e.g.

{pasta}	support = 4
{lemon}	support = 3
{bread}	support = 1
{orange}	support = 3
{cake}	support = 2

The Apriori algorithm

Step 2: eliminate infrequent itemsets.

e.g.

{pasta} support = 4

{lemon} support = 3

{bread} support = 1

{orange} support = 3

{cake} support = 2

The Apriori algorithm

Step 2: eliminate infrequent itemsets.

e.g.

{pasta}	support = 4
{lemon}	support = 3
{orange}	support = 3
{cake}	support = 2

The Apriori algorithm

Step 3: generate candidates of size 2 by combining pairs of frequent itemsets of size 1.

Frequent items

{pasta}
{lemon}
{orange}
{cake}



Candidates of size 2

{pasta, lemon}
{pasta, orange}
{pasta, cake}
{lemon, orange}
{lemon, cake}
{orange, cake}

The Apriori algorithm

Step 4: Eliminate candidates of size 2 that have an infrequent subset (Property 2)

(none!)

Frequent items

{pasta}
{lemon}
{orange}
{cake}

Candidates of size 2

{pasta, lemon}
{pasta, orange}
{pasta, cake}
{lemon, orange}
{lemon, cake}
{orange, cake}

The Apriori algorithm

Step 5: scan the database to calculate the support of remaining candidate itemsets of size 2.

Candidates of size 2

{pasta, lemon} support: 3

{pasta, orange} support: 3

{pasta, cake} support: 2

{lemon, orange} support: 2

{lemon, cake} support: 1

{orange, cake} support: 2

The Apriori algorithm

Step 6: eliminate infrequent candidates of size 2

Candidates of size 2

{pasta, lemon} support: 3

{pasta, orange} support: 3

{pasta, cake} support: 2

{lemon, orange} support: 2

~~{lemon, cake} support: 1~~

{orange, cake} support: 2

The Apriori algorithm

Step 6: eliminate infrequent candidates of size 2

Frequent itemsets of size 2

{pasta, lemon} support: 3

{pasta, orange} support: 3

{pasta, cake} support: 2

{lemon, orange} support: 2

{orange, cake} support: 2

The Apriori algorithm

Step 7: generate candidates of size 3 by combining frequent pairs of itemsets of size 2.

Frequent itemsets of size 2

{pasta, lemon}

{pasta, orange}

{pasta, cake}

{lemon, orange}

{orange, cake}



Candidates of size 3

{pasta, lemon, orange}

{pasta, lemon, cake}

{pasta, orange, cake}

{lemon, orange, cake}

The Apriori algorithm

Step 8: eliminate candidates of size 3 having a subset of size 2 that is infrequent.

Frequent itemsets of size 2

{pasta, lemon}

{pasta, orange}

{pasta, cake}

{lemon, orange}

{orange, cake}

Candidates of size 3

{pasta, lemon, orange}

~~{pasta, lemon, cake}~~

{pasta, orange, cake}

~~{lemon, orange, cake}~~

Because {lemon, cake} is infrequent!

The Apriori algorithm

Step 8: eliminate candidates of size 3 having a subset of size 2 that is infrequent.

Frequent itemsets of size 2

{pasta, lemon}

{pasta, orange}

{pasta, cake}

{lemon, orange}

{orange, cake}

Candidates of size 3

{pasta, lemon, orange}

{pasta, orange, cake}

Because {lemon, cake} is infrequent!

The Apriori algorithm

Step 9: scan the database to calculate the support of the remaining candidates of size 3.

Candidates of size 2

{pasta, lemon, orange} support: 2

{pasta, orange, cake} support: 2

The Apriori algorithm

Step 10: eliminate infrequent candidates (none!)

frequent itemsets of size 3

{pasta, lemon, orange} support: 2

{pasta, orange, cake} support: 2

The Apriori algorithm

Step 11: generate candidates of size 4 by combining pairs of frequent itemsets of size 3.

Frequent itemsets of size 3

{pasta, lemon, orange}

{pasta, orange, cake}



Candidates of size 4

{pasta, lemon, orange, cake}

The Apriori algorithm

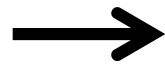
Step 12: eliminate candidates of size 4 having a subset of size 3 that is infrequent.

Frequent itemsets of size 3

{pasta, lemon, orange}

{pasta, orange, cake}

Candidates of size 4



~~{pasta, lemon, orange, cake}~~

The Apriori algorithm

Step 12: Since there is no more candidates, we cannot generate candidates of size 5 and the algorithm stops.

Candidates of size 4

~~{pasta, lemon, orange, cake}~~

Result →

Final result

{pasta}	support = 4
{lemon}	support = 3
{orange}	support = 3
{cake}	support = 2
{pasta, lemon}	support: 3
{pasta, orange}	support: 3
{pasta, cake}	support: 2
{lemon, orange}	support: 2
{orange, cake}	support: 2
{pasta, lemon, orange}	support: 2
{pasta, orange, cake}	support: 2

Technical details

Combining different itemsets can generate the same candidate.

Example:

$\{A, B\}$ and $\{A, E\} \rightarrow \{A, B, E\}$

$\{B, E\}$ and $\{A, E\} \rightarrow \{A, B, E\}$

problem: some candidates are generated several times!

Technical details

Combining different itemsets can generate the same candidate.

Example:

$\{A, B\}$ and $\{A, E\} \rightarrow \{A, B, E\}$

$\{B, E\}$ and $\{A, E\} \rightarrow \{A, B, E\}$

Solution:

- Sort items in each itemsets (e.g. by alphabetical order)
 - Combine two itemsets only if all items are the same except the last one.
-

Apriori vs the naïve algorithm

- The Apriori property can considerably reduce the number of itemsets to be considered.
- In the previous example:
 - Naïve approach:
 $2^5 - 1 = 31$ itemsets are considered
 - By using the Apriori property:
18 itemsets are considered

How to evaluate this type of algorithms?

- ☐ Execution time,
 - ☐ Memory used,
 - ☐ *Scalability*: how the performance is influenced by the number of transactions
 - ☐ Performance on different types of data:
 - real data,
 - synthetic (fake) data,
 - dense vs sparse data,...
 - ☐ ...
-

Performance of Apriori

The performance of Apriori depends on several factors:

- ❑ the *minsup* parameter: the more it is set low, the larger the search space and the number of itemsets will be.
 - ❑ the number of items,
 - ❑ the number of transactions,
 - ❑ The average transaction length.
-

Problems of Apriori

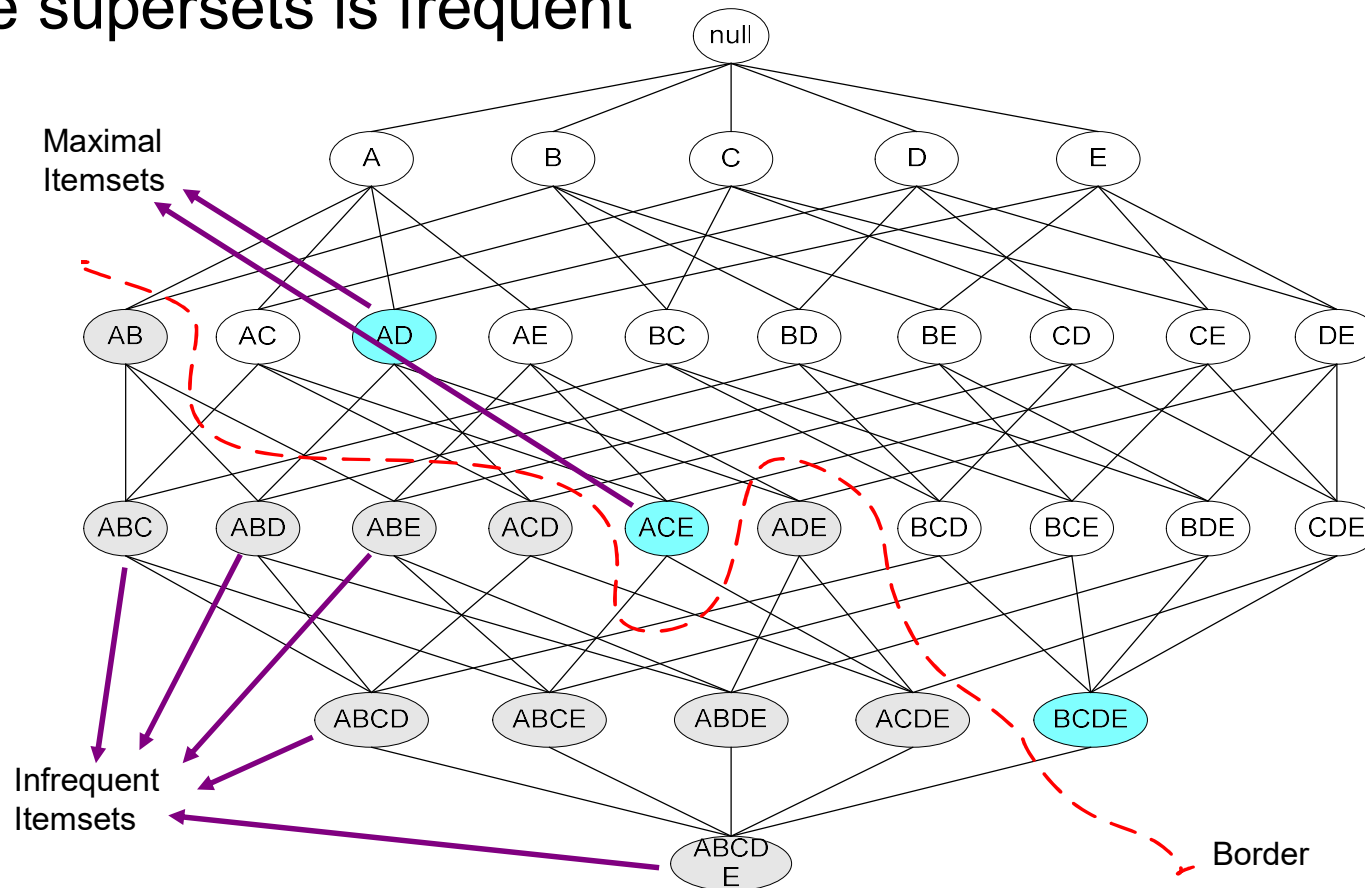
- ☐ can generate numerous candidates
 - ☐ requires to scan the database numerous times.
 - ☐ candidates may not exist in the database.
 - ☐ ...
-

Factors Affecting Complexity of Apriori

- ❑ Choice of minimum support threshold
 - lowering support threshold results in more frequent itemsets
 - this may increase number of candidates and max length of frequent itemsets
- ❑ Dimensionality (number of items) of the data set
 - More space is needed to store support count of itemsets
 - if number of frequent itemsets also increases, both computation and I/O costs may also increase
- ❑ Size of database
 - run time of algorithm increases with number of transactions
- ❑ Average transaction width
 - transaction width increases the max length of frequent itemsets
 - number of subsets in a transaction increases with its width, increasing computation time for support counting
- ❑ Solution: Concise Itemset Representation
 - ➔ Mine *closed patterns* and *max-patterns*

Maximal Frequent Itemset

An itemset X is maximal frequent if it is frequent and none of its immediate supersets is frequent



An illustrative example

		Items									
Transactions		A	B	C	D	E	F	G	H	I	J
	1										
	2										
	3										
	4										
	5										
	6										
	7										
	8										
	9										
	10										

Support threshold (by count) : 5

Frequent itemsets: ?

Maximal itemsets: ?

An illustrative example

		Items									
		A	B	C	D	E	F	G	H	I	J
Transactions	1										
	2										
	3										
	4										
	5										
	6										
	7										
	8										
	9										
	10										

Support threshold (by count) : 5

Frequent itemsets: {F}

Maximal itemsets: {F}

Support threshold (by count): 4

Frequent itemsets: ?

Maximal itemsets: ?

An illustrative example

Transactions	Items									
	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

Support threshold (by count) : 5

Frequent itemsets: {F}

Maximal itemsets: {F}

Support threshold (by count): 4

Frequent itemsets: {E}, {F}, {E,F}, {J}

Maximal itemsets: {E,F}, {J}

Support threshold (by count): 3

Frequent itemsets: ?

Maximal itemsets: ?

An illustrative example

Transactions	Items									
	A	B	C	D	E	F	G	H	I	J
	1									
	2									
	3									
	4									
	5									
	6									
	7									
	8									
	9									
	10									

Support threshold (by count) : 5

Frequent itemsets: {F}

Maximal itemsets: {F}

Support threshold (by count): 4

Frequent itemsets: {E}, {F}, {E,F}, {J}

Maximal itemsets: {E,F}, {J}

Support threshold (by count): 3

Frequent itemsets:

All subsets of {C,D,E,F} + {J}

Maximal itemsets:

{C,D,E,F}, {J}

Another illustrative example

	Items									
	A	B	C	D	E	F	G	H	I	J
Transactions	1									
	2									
	3									
	4									
	5									
	6									
	7									
	8									
	9									
	10									

Support threshold (by count) : 5

Maximal itemsets: {A}, {B}, {C}

Support threshold (by count): 4

Maximal itemsets: {A,B}, {A,C},{B,C}

Support threshold (by count): 3

Maximal itemsets: {A,B,C}

Closed Itemset

- ❑ An itemset X is closed if none of its immediate supersets has the same support as the itemset X .
- ❑ X is not closed if at least one of its immediate supersets has support count as X .

TID	Items
1	{A,B}
2	{B,C,D}
3	{A,B,C,D}
4	{A,B,D}
5	{A,B,C,D}

Itemset	Support
{A}	4
{B}	5
{C}	3
{D}	4
{A,B}	4
{A,C}	2
{A,D}	3
{B,C}	3
{B,D}	4
{C,D}	3

Itemset	Support
{A,B,C}	2
{A,B,D}	3
{A,C,D}	2
{B,C,D}	2
{A,B,C,D}	2

Example 1

Transactions	Items									
	A	B	C	D	E	F	G	H	I	J
	1									
	2									
	3									
	4									
	5									
	6									
	7									
	8									
	9									
	10									

Itemsets	Support (counts)	Closed itemsets
{C}	3	✓
{D}	2	
{C,D}	2	✓

Example 2

		Items									
Transactions		A	B	C	D	E	F	G	H	I	J
	1										
	2										
	3										
	4										
	5										
	6										
	7										
	8										
	9										
	10										

Itemsets	Support (counts)	Closed itemsets
{C}	3	
{D}	2	
{E}	2	
{C,D}	2	
{C,E}	2	
{D,E}	2	
{C,D,E}	2	

Example 2

	Items									
	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

Itemsets	Support (counts)	Closed itemsets
{C}	3	✓
{D}	2	
{E}	2	
{C,D}	2	
{C,E}	2	
{D,E}	2	
{C,D,E}	2	✓

Example 3

		Items									
Transactions		A	B	C	D	E	F	G	H	I	J
	1										
	2										
	3										
	4										
	5										
	6										
	7										
	8										
	9										
	10										

Closed itemsets: {C,D,E,F}, {C,F}

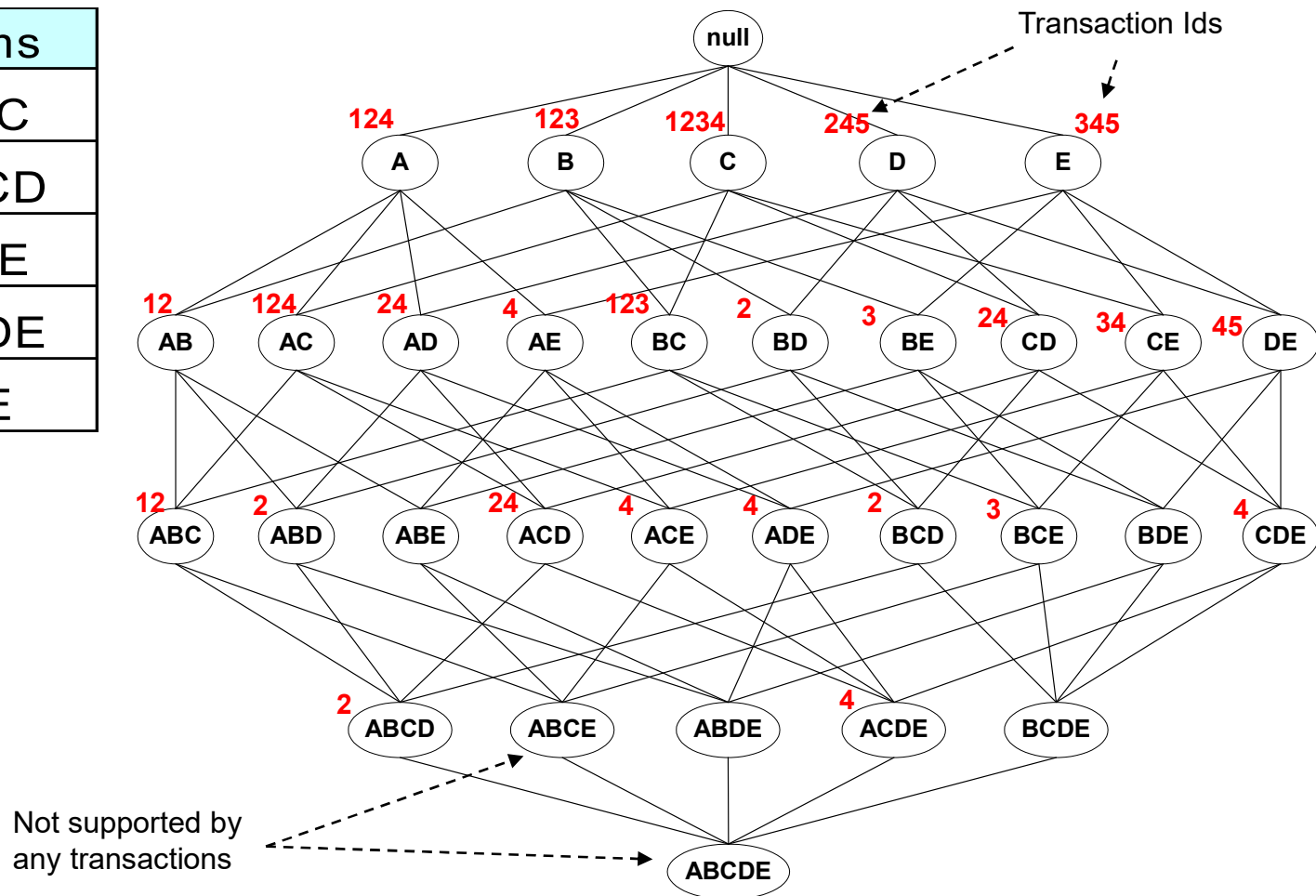
Example 4

		Items									
Transactions		A	B	C	D	E	F	G	H	I	J
	1										
	2										
	3										
	4										
	5										
	6										
	7										
	8										
	9										
	10										

Closed itemsets: {C,D,E,F}, {C}, {F}

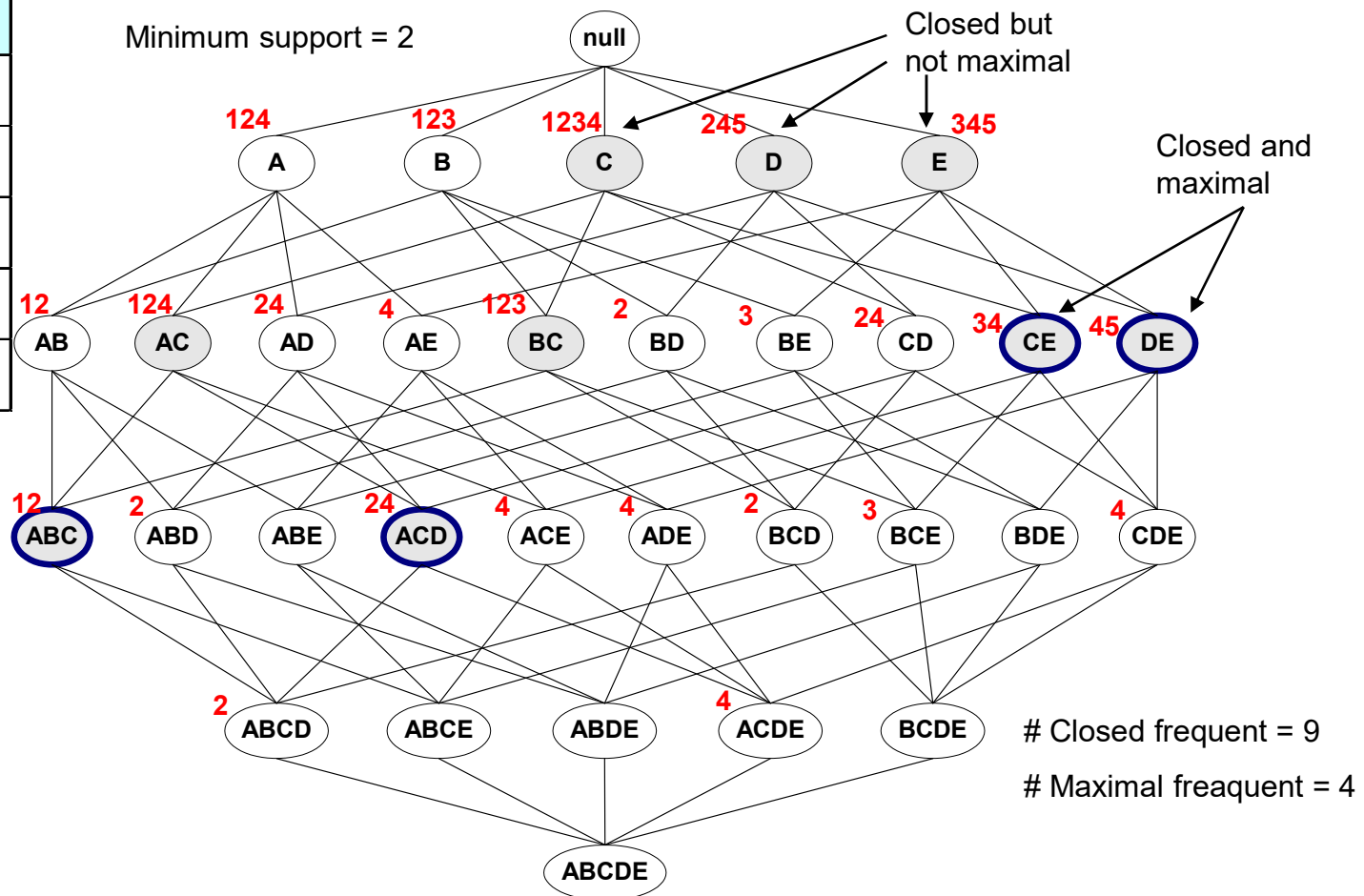
Maximal vs Closed Itemsets

TID	Items
1	ABC
2	ABCD
3	BCE
4	ACDE
5	DE

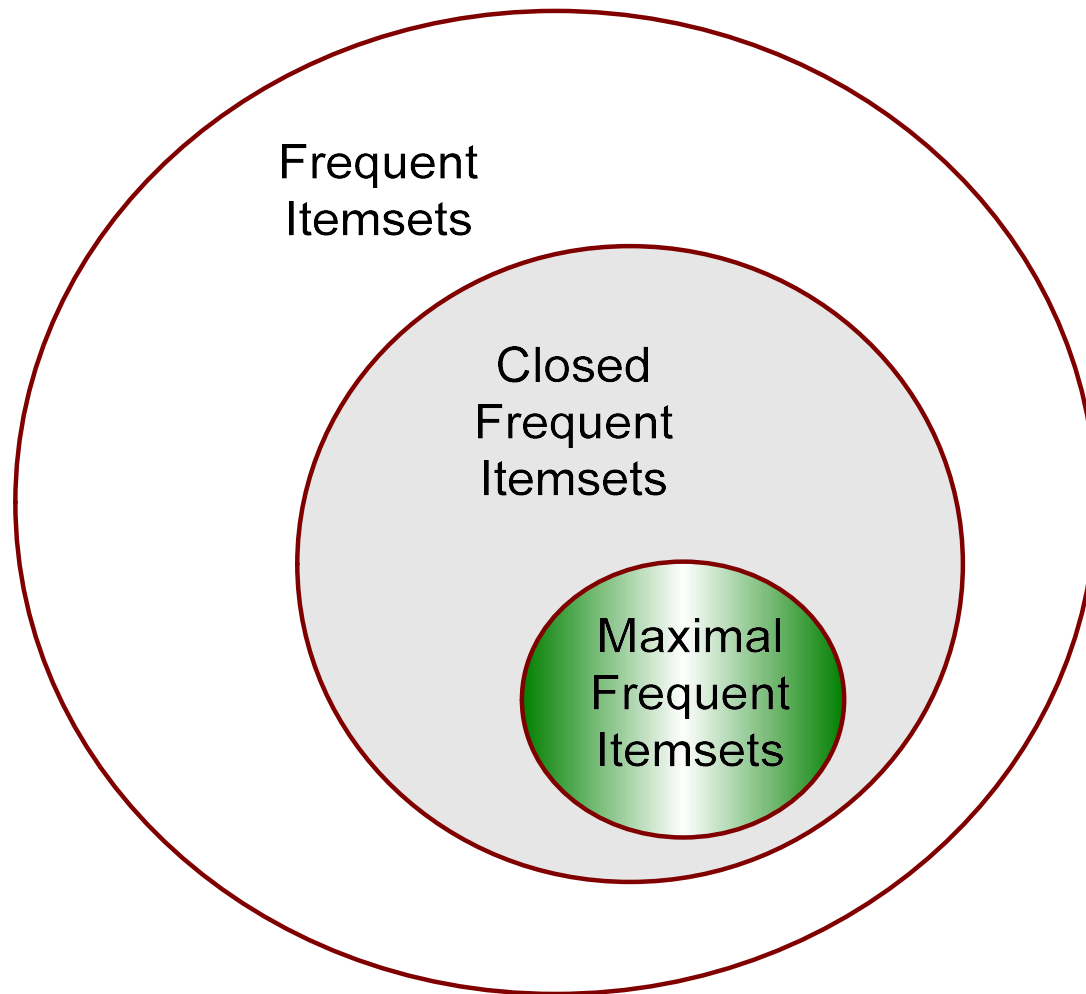


Maximal Frequent vs Closed Frequent Itemsets

TID	Items
1	ABC
2	ABCD
3	BCE
4	ACDE
5	DE

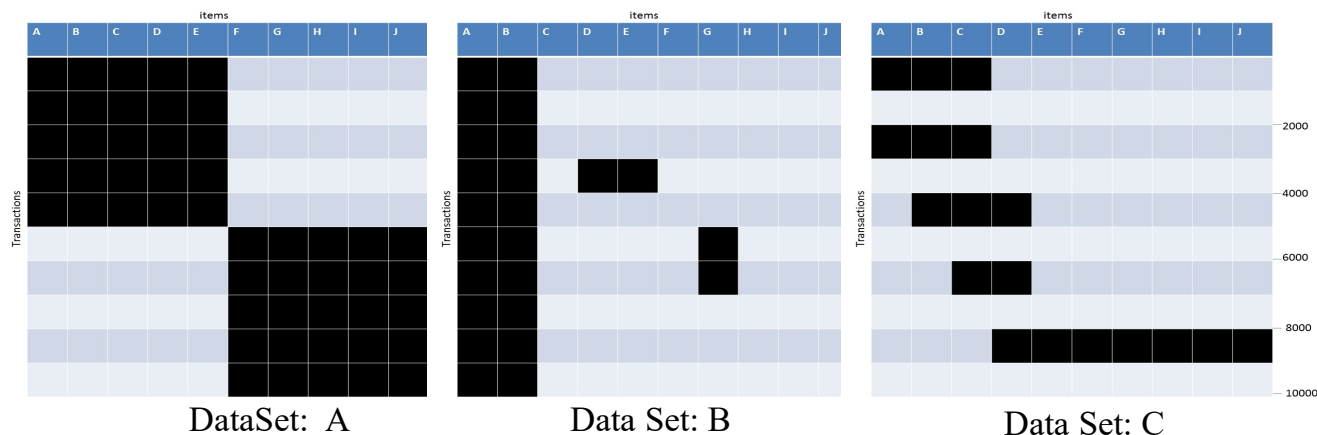


Maximal vs Closed Itemsets



Example question

- Given the following transaction data sets (dark cells indicate presence of an item in a transaction) and a support threshold of 20%, answer the following questions



- What is the number of frequent itemsets for each dataset? Which dataset will produce the most number of frequent itemsets?
- Which dataset will produce the longest frequent itemset?
- Which dataset will produce frequent itemsets with highest maximum support?
- Which dataset will produce frequent itemsets containing items with widely varying support levels (i.e., itemsets containing items with mixed support, ranging from 20% to more than 70%)?
- What is the number of maximal frequent itemsets for each dataset? Which dataset will produce the most number of maximal frequent itemsets?
- What is the number of closed frequent itemsets for each dataset? Which dataset will produce the most number of closed frequent itemsets?

Exercise

Let a transaction database as:

1. Apply **Apriori** algorithm to find all frequent itemsets with **minsupp = 22%**
3. List all maximal frequent itemsets and closed frequent itemsets
3. Find all association rules with:
 - a. **Minconf = 50 %**
 - b. **Minconf = 70%**

Tid	Items
100	M1, M2, M5
200	M2, M4
300	M2, M3
400	M1, M2, M4
500	M1, M3
600	M2, M3
700	M1, M3
800	M1, M2, M3, M5
900	M1, M2, M3